

Month 3: Purple Team Batch 01

Web App, API, & Cloud Security

Week 09: API Security Fundamentals & Enumeration

Instructor: **Muhammad Saad**

REST (Representational State Transfer):

REST is an architectural style used for designing networked applications, where communication happens over HTTP using simple operations like GET, POST, PUT, and DELETE, usually returning data in formats like JSON or XML.

Think of REST like ordering food at a restaurant 🍔

- You tell the waiter (API) what you want using simple rules (like GET, POST).
- The waiter brings you exactly what you asked for, usually in a simple format like JSON.
- It's lightweight, fast, and easy to use.

In short: **REST = simple, fast way for apps to talk over the internet**

SOAP (Simple Object Access Protocol):

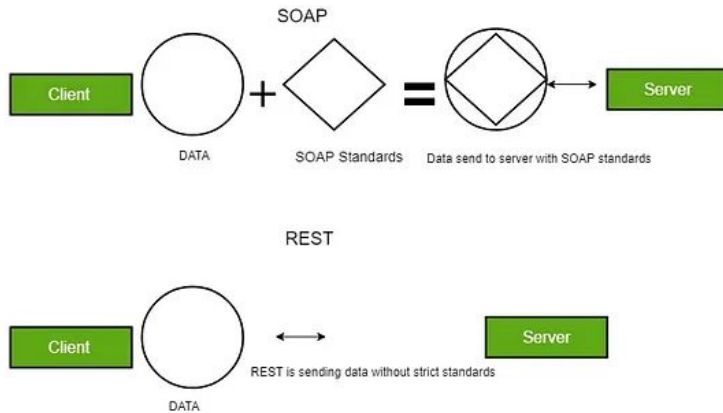
SOAP is a protocol used for exchanging structured information between applications, which relies on XML format and follows strict rules for messaging, security, and communication.

SOAP is more like sending a formal letter 📄

- Everything must follow strict rules and structure (XML format only).
- It includes extra details like security and error handling built-in.
- It's heavier and more complex.

In short: **SOAP = strict, structured, and secure way for apps to communicate**

SOAP vs REST Web Services



Use Cases of SOAP:

Banking Systems: Secure transactions and sensitive financial data.

Enterprise Applications: Large systems (like ERP/CRM) where strict rules are required.

Payment Gateways: Systems that require built-in security and reliability.

Use Cases of REST:

Mobile & Web Apps: Apps like social media or e-commerce use REST APIs to fetch data quickly (e.g., user profiles, posts).

Public APIs: Services like weather APIs or maps use REST because it's lightweight and easy for developers.

JSON (JavaScript Object Notation)

JSON is a lightweight data format used to store and exchange data between systems. It is easy to read and write, and it uses a structure similar to key–value pairs.

```
</> JSON
```

```
{  
  "name": "Ali",  
  "age": 20  
}
```

In short: **JSON = simple, lightweight, and easy-to-read data format**

XML (eXtensible Markup Language)

XML is a data format that uses tags to store and transport data. It is more structured and strict compared to JSON.

Example:

```
</> XML
```

```
<person>  
  <name>Ali</name>  
  <age>20</age>  
</person>
```

In short: **XML = structured, tag-based data format used for storing and sharing data**

Broken Access Control?

Broken Access Control is a security vulnerability where an application does not properly restrict what users are allowed to do. This means a user can access data or perform actions that they should not be allowed to.

In short: **Broken Access Control = users can access or do things beyond their permissions**

1. IDOR (Insecure Direct Object Reference)

IDOR happens when an application directly uses user input (like an ID in a URL) to access objects (files, records, etc.) without checking if the user is authorized.

Example:

A URL like: `/profile?id=123`

If a user changes it to: `/profile?id=124`

and can see someone else's data, that's IDOR.

In short: **IDOR = direct access to data by changing IDs without permission checks**

2. BOLA (Broken Object Level Authorization)

BOLA is similar to IDOR but more general and common in APIs. It happens when an application fails to verify if a user has permission to access a specific object (like a user account or record).

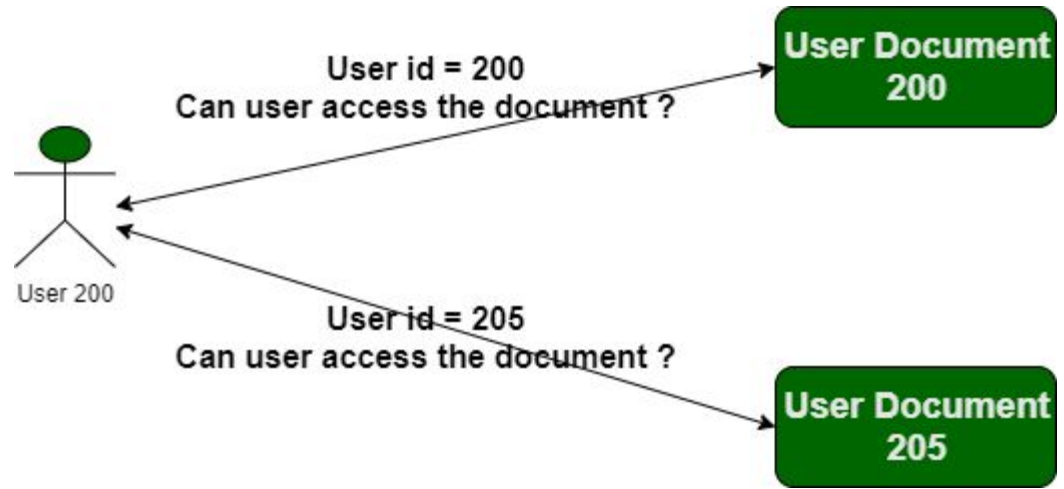
Example:

An API request: `GET /api/user/123`

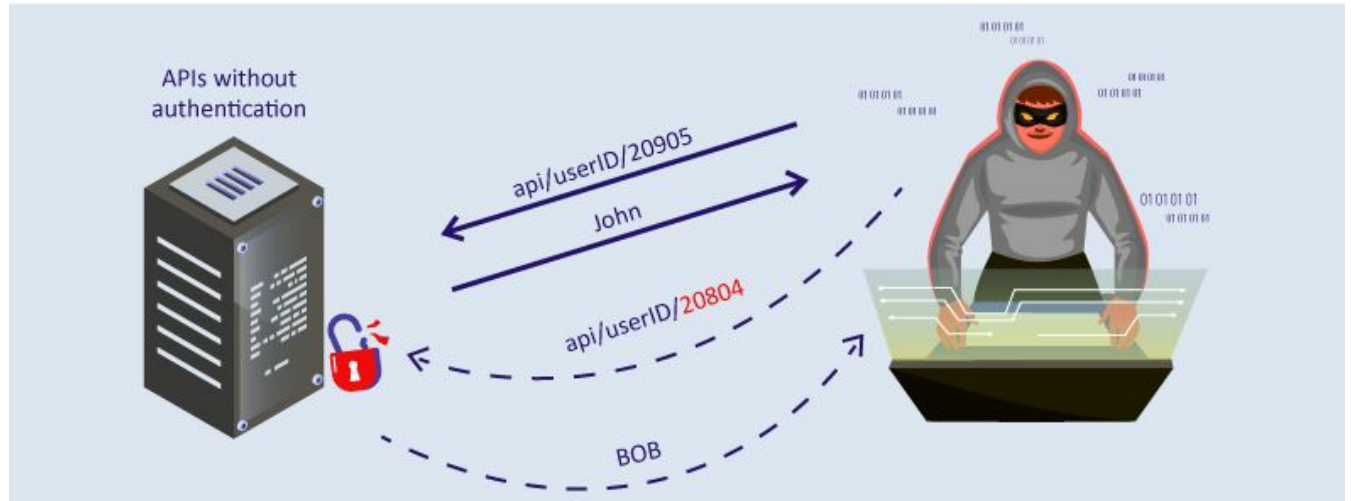
If a user can change `123` to another user's ID and get their data, it's BOLA.

In short: **BOLA = missing authorization checks at object level (especially in APIs)**

IDOR



BOLA



How to Prevent IDOR / BOLA

Enforce Object-Level Authorization (MOST IMPORTANT)

Always check if the logged-in user has permission to access the object.

Example logic:

- User requests `/api/users/102`
- Server verifies: **Does user 102 belong to this logged-in user?**
- If NO → return `403 Forbidden`

Enforce Object-Level Authorization (MOST IMPORTANT)

Never trust user-supplied IDs alone.

Instead of relying only on: `/users/101`

Verify using session: `session.user_id == requested.user_id`

Implement Role-Based Access Control (RBAC)

Define clear rules:

- Admin → can access all data
- User → only their own data

Avoid Direct Object References (or limit them)

Instead of exposing raw IDs:

- Use indirect references (UUIDs, tokens)
- Or map internal IDs securely

Example:

Instad of `/user/101`

Use `/user/profile` (server decides whose profile it is)

Best of Luck for the Week 9 Tasks

Instructor: **Muhammad Saad**